

Tema 11: Ficheros ELF, Enlazado y Depuracion

Este tema cubre el formato de los ejecutables en Unix, como se enlazan las bibliotecas, y las herramientas de depuracion. Aparece con frecuencia en el examen.

Ficheros ELF

ELF significa Executable and Linkable Format, y es el formato estandar para ejecutables, bibliotecas y ficheros objeto en Linux y Unix. Un fichero ELF contiene varias secciones, cada una con un proposito distinto.

La seccion punto text contiene el codigo maquina, es decir, las instrucciones del programa. La seccion punto data contiene las variables globales inicializadas. La seccion punto bss contiene las variables globales no inicializadas, que se ponen a cero al cargar el programa. La seccion punto got, que es la Global Offset Table, y punto plt, que es la Procedure Linkage Table, se usan para el enlazado dinamico.

Pregunta de examen: la seccion de un fichero ELF que contiene las instrucciones del programa es punto text.

Un fichero ELF tambien contiene informacion sobre la arquitectura del procesador necesario, el punto de entrada del programa, informacion de enlazado, y los datos inicializados. Lo que no contiene es informacion sobre el planificador. El tipo de planificador lo decide el kernel en tiempo de ejecucion, no el ejecutable.

Enlazado estatico y dinamico

El enlazado estatico copia todo el codigo de las bibliotecas dentro del ejecutable. El resultado es un fichero grande pero autocontenido que no depende de bibliotecas externas.

El enlazado dinamico no copia el codigo de las bibliotecas. En su lugar, anota las dependencias en el ejecutable, y las bibliotecas se cargan en memoria en tiempo de ejecucion. La ventaja es que los ejecutables son mas pequeños y las bibliotecas se comparten entre procesos.

Lazy binding

Lazy binding es una optimizacion del enlazado dinamico. En vez de resolver la direccion de todas las funciones de biblioteca al iniciar el programa, las resuelve la primera vez que se llaman. Esto acelera el inicio del programa, especialmente si usa muchas funciones de biblioteca pero solo llama a unas pocas.

Internamente funciona asi: la primera vez que llamas a una funcion de biblioteca, el programa salta a la PLT, que redirige al enlazador dinamico. El enlazador encuentra la direccion real de la funcion, la escribe en la GOT, y salta a la funcion. Las siguientes llamadas van directamente a la GOT, que ya tiene la direccion correcta.

Pregunta de examen: con lazy binding, la direccion de una funcion no se resuelve hasta la primera llamada.

RELRO

RELRO significa Relocation Read-Only. Es un modo de seguridad que resuelve todas las reubicaciones de simbolos al inicio del programa, antes de empezar a ejecutar, y luego marca las secciones como de solo lectura para prevenir ataques que sobrescriban las entradas de la GOT.

Pregunta de examen: con RELRO, todos los simbolos de las bibliotecas se resuelven cuando el programa comienza a ejecutar, antes del main.

El comando ldd

Ldd muestra las bibliotecas dinamicas que necesita un ejecutable ELF. Si ejecutas ldd barra bin barra cat, te muestra que necesita libc, el enlazador dinamico, y cualquier otra dependencia. Es util para diagnosticar problemas de bibliotecas faltantes.

Pregunta de examen: ldd muestra las bibliotecas dinamicas que necesita un binario ELF, no las estaticas.

GDB: el depurador

GDB es el depurador de GNU para programas en C. Te permite ejecutar un programa paso a paso, poner puntos de ruptura, inspeccionar variables y ver la pila de llamadas.

Los comandos principales son: run para ejecutar el programa, break para poner un punto de ruptura en una funcion o linea, continue para continuar la ejecucion hasta el siguiente punto de ruptura, step para ejecutar una linea entrando en las funciones llamadas, next para ejecutar una linea sin entrar en funciones, print para mostrar el valor de una variable, backtrace para ver la pila de llamadas, info locals para ver las variables locales, y quit para salir.

Para usar GDB, necesitas compilar con la opcion menos g, que incluye informacion de depuracion en el ejecutable. Sin esta opcion, GDB no puede mostrar nombres de variables ni numeros de linea.

Valgrind: deteccion de errores de memoria

Valgrind es una herramienta que ejecuta tu programa en un entorno simulado y detecta automaticamente errores de memoria. Los errores que detecta incluyen: uso de memoria no inicializada, lectura o escritura fuera de los limites de un bloque asignado, uso de memoria despues de liberarla, y memory leaks.

Para usarlo, ejecutas valgrind menos menos leak-check igual full seguido de tu programa. Valgrind te mostrara un informe con todos los errores encontrados y un resumen de fugas de memoria.

Un informe limpio de Valgrind muestra: cero errores, cero bytes perdidos, y el mismo numero de malloc que de free. Si tienes mas mallocs que frees, tienes fugas. Si tienes accesos a memoria invalida, probablemente tienes punteros mal usados.

Compilacion con multiples ficheros

Cuando un proyecto crece, se divide en multiples ficheros punto c y punto h. Los ficheros punto c contienen la implementacion, y los ficheros punto h contienen las declaraciones, es decir, los prototipos de funciones.

Las reglas importantes son: una variable global debe estar definida en un unico fichero punto c. Si otros ficheros la necesitan, la declaran con la palabra clave extern. Las funciones y variables static no son visibles desde otros ficheros, lo que se llama enlazado interno.

Los ficheros punto h deben tener guardias de inclusion para evitar ser incluidos dos veces. La forma estandar es: almohadilla ifndef NOMBRE H, almohadilla define NOMBRE H, contenido, y almohadilla endif. Esto evita errores de redefinicion.

Para compilar multiples ficheros: primero compilas cada fichero punto c por separado con la opcion menos c para generar los punto o, y luego los enlazas todos juntos con gcc para generar el ejecutable final.